

TECHNICAL DOCUMENTATION TD-001

DOCUMENTBUILDER: HOW IT WORKS

Understanding the 'class that knows it has this ability'

PUBLIC

Issued by: WAFT Documentation Team

Date: January 11, 2026

DocumentBuilder: How It Works

Understanding the "class that knows it has this ability"

CONTRIBUTORS

This document was created through collaborative work between:

- **Human:** Christopher Tavalazzi
 - **Claude Opus 4.5:** Cloud-based AI assistant (Claude Code Cloud Browser Chat environment)
 - **Cursor System:** Cursor system using multiple AI LLMs available at the time of release of Claude Opus 4.5, Gemini 3, GPT 5.2, and others
-

THE CORE CONCEPT

DocumentBuilder is designed as a **self-aware document generator** - a class that knows it can generate documents and provides a simple, fluent API for doing so.

PUBLIC

HOW IT "KNOWS" IT CAN GENERATE DOCUMENTS

1. Template Awareness

The class knows what templates it can use:

```
class TemplateType(Enum):  
    """Available document templates."""  
  
    FIELD_GUIDE = "field_guide"  
  
    LAB_NOTES = "lab_notes"  
  
    TM_REPORT = "tm_report"  
  
    # ... 12 total templates
```

What this means: The class has explicit knowledge of its capabilities through the `TemplateType` enum. It's not guessing - it knows exactly what it can do.

2. Configuration Encapsulation

Each document is configured through `DocumentConfig`:

```
@dataclass  
class DocumentConfig:  
  
    template: TemplateType          # What template to use  
  
    title: str                      # Document title  
  
    content: str                    # HTML content
```

```
printer_friendly: bool = False # Can convert to printer-friendly

# ... other options
```

What this means: The class encapsulates all the information needed to generate a document. It "knows" what it needs to know.

3. Fluent API Design

The class provides factory methods that return configured instances:

The class "knows" it can create field guides

```
doc = DocumentBuilder.field_guide(

    title="My Guide",

    content="
```

INTRO

```
Content

"

)
```

The instance "knows" it can generate itself

```
doc.save("output.pdf") # Generates the PDF
```

What this means: The API is self-documenting. Calling `DocumentBuilder.field_guide()` immediately tells you "this class knows how to create field guides."

4. Automatic Capabilities

The class automatically handles:

- Template selection (knows which template to use)
- Printer-friendly conversion (knows how to convert)
- PDF generation (knows how to render)
- Path management (knows where to save)

```
def generate(self, output_path: Optional[Path] = None) -> Path:
    """Generate the PDF document."""

    # 1. Knows which template to get

    template_str = self._get_template()

    # 2. Knows how to convert if needed

    if self.config.printer_friendly:

        template_str = convert_html_template_to_printer_friendly(template_str)

    # 3. Knows how to render

    template = Template(template_str)
```

```
html_output = template.render(...)
```

```
# 4. Knows how to generate PDF
```

```
HTML(string=html_output).write_pdf(output_path)
```

```
return output_path
```

THE "SELF-AWARENESS" PATTERN

What Makes It Self-Aware?

- **Explicit Capability Declaration**

- The class declares what it can do through `TemplateType` enum
- Factory methods (`field_guide()`, `lab_notes()`) make capabilities discoverable
- No guessing - you can see what it can do

- **Encapsulated Knowledge**

- The class knows how to:
 - Select templates (`_get_template()`)
 - Convert to printer-friendly
 - Render HTML with Jinja2
 - Generate PDFs with WeasyPrint
- All this knowledge is encapsulated in the class

- **Fluent Interface**

- The API guides you: `DocumentBuilder.field_guide(...).save(...)`
- Each step is clear: create → configure → generate
- The class "knows" the workflow

- **Composition Awareness**

- The class knows it can be part of collections:

```
collection = DocumentBuilder.collection("My Project")

collection.add(doc) # DocumentBuilder knows it can be added

collection.save("booklet.pdf") # Knows how to create binders
```

HOW IT WORKS INTERNALLY

Step 1: Creation

```
doc = DocumentBuilder.field_guide(
    title="My Guide",

    content="
```

INTRO

```
"
```

```
)
```

What happens:

- Factory method creates a DocumentConfig with template=TemplateType.FIELD_GUIDE
- Returns a DocumentBuilder instance with that config
- The instance "knows" it's a field guide

Step 2: Configuration (Optional)

```
doc = DocumentBuilder.field_guide(  
    title="My Guide",  
  
    content="
```

INTRO

```
",  
  
    printer_friendly=True, # Instance knows it should convert  
  
    series="MANUAL",  
  
    number="M-001"  
  
)
```

What happens:

- Config is set with all options
- Instance "knows" it needs printer-friendly conversion
- Instance "knows" its series and number

Step 3: Generation

```
doc.save("output.pdf")
```

What happens:

- `save()` calls `generate()`
- `generate()` calls `_get_template()` - **knows which template**
- If `printer_friendly=True`, converts template - **knows how to convert**
- Renders with Jinja2 - **knows how to render**

- Generates PDF with WeasyPrint - **knows how to create PDF**
 - Returns path - **knows where it saved**
-

THE "KNOWING" MECHANISM

Explicit Knowledge (What It Knows)

- **Template Knowledge**

```
def _get_template(self) -> str:
    if self.config.template == TemplateType.FIELD_GUIDE:

        return FIELD_GUIDE_TEMPLATE
```

Knows what to return for each type

- **Conversion Knowledge**

```
if self.config.printer_friendly:

    template_str = convert_html_template_to_printer_friendly(template_str)
```

Knows how to convert

- **Rendering Knowledge**

```
template = Template(template_str) # Knows Jinja2
html_output = template.render(...) # Knows how to render
```

- **PDF Generation Knowledge**

```
HTML(string=html_output).write_pdf(output_path) # Knows WeasyPrint
```


Implicit Knowledge (What It Assumes)

- It assumes WeasyPrint is available
 - It assumes templates are valid HTML/CSS
 - It assumes content is valid HTML
 - It assumes output directory can be created
-

COMPARISON: BEFORE VS. AFTER

Before (Not Self-Aware)

Multiple functions, no unified class

```
from src.waft.templates.field_guide import generate_field_guide
```

```
from src.waft.templates.lab_notes import generate_lab_notes
```

```
from scripts.printer_friendly_helper import convert_css_to_printer_friendly
```

Manual process - you have to know everything

```
content = "
```

INTRO

```
"
```

```
template = get_template() # You get it
```

```
if printer_friendly:

    template = convert(template) # You convert it

generate_field_guide(...) # You call it
```

Problem: You have to know:

- Which function to call
- How to convert to printer-friendly
- What parameters each function needs
- How to combine multiple documents

After (Self-Aware)

Single class, knows everything

```
from waft import DocumentBuilder
```

Class knows what to do

```
DocumentBuilder.field_guide(

    title="My Guide",

    content="
```

INTRO

",

```
printer_friendly=True # Class knows how to handle this
```

```
).save("output.pdf") # Class knows how to generate
```

Solution: The class knows:

- Which template to use
 - How to convert to printer-friendly
 - What parameters are needed
 - How to generate the PDF
 - How to work with collections
-

THE DESIGN PHILOSOPHY

Principle 1: Encapsulation of Knowledge

The class encapsulates all knowledge about document generation:

- Template selection logic
- Conversion logic
- Rendering logic
- PDF generation logic

Result: You don't need to know these details - the class knows them.

Principle 2: Discoverable Capabilities

Capabilities are discoverable through the API:

- `DocumentBuilder.field_guide()` - clearly a field guide capability
- `DocumentBuilder.lab_notes()` - clearly a lab notes capability
- `DocumentBuilder.collection()` - clearly a collection capability

Result: You can discover what the class can do by looking at its methods.

Principle 3: Fluent Interface

The API guides you through the process:

```
DocumentBuilder.field_guide(...) # Step 1: Create  
    .save(...)                  # Step 2: Generate
```

Result: The workflow is self-evident.

Principle 4: Composition

The class knows it can be composed:

```
collection = DocumentBuilder.collection("Project")  
collection.add(DocumentBuilder.field_guide(...)) # Knows it can be added  
  
collection.save("booklet.pdf") # Knows how to create binder
```

Result: Complex documents are built from simple parts.

REAL-WORLD EXAMPLE

What the Class "Knows" in This Example

Create a document

```
doc = DocumentBuilder.field_guide(  
  
    title="WAFT Status Report",  
  
    content="
```

CURRENT STATE

System is healthy.

",

```
printer_friendly=True,
```

```
series="STATUS REPORT",
```

```
number="SR-001"
```

```
)
```

The class "knows":

1. It's a field guide (from field_guide() method)
2. It should use FIELD_GUIDE template
3. It should convert to printer-friendly
4. It has series "STATUS REPORT" and number "SR-001"

5. It can generate itself when save() is called

```
doc.save("status.pdf")
```

When save() is called, the class "knows" to:

1. Get the field guide template
2. Convert it to printer-friendly (because flag
3. Render it with the provided content
4. Generate a PDF at "status.pdf"

WHY THIS MATTERS

For Users

Before: You had to know:

- Which function to import
- How to convert templates
- What parameters each function needs

- How to combine documents

After: You just know:

- `DocumentBuilder.field_guide(...).save(...)`
- The class handles everything else

For Developers

Before: Document generation logic was scattered:

- In template files
- In helper scripts
- In example scripts
- Hard to find, hard to reuse

After: Everything is in one place:

- `DocumentBuilder` class
 - Clear API
 - Easy to extend
 - Easy to test
-

EXTENDING THE "KNOWLEDGE"

Adding a New Template Type

The class can "learn" new capabilities:

1. Add to enum (declares capability)

```
class TemplateType(Enum):
```

```
    # ... existing
```

```
    NEW_TEMPLATE = "new_template"
```

2. Add factory method (makes it discoverable)

```
@classmethod

def new_template(cls, ...) -> "DocumentBuilder":

    config = DocumentConfig(template=TemplateType.NEW_TEMPLATE, ...)

    return cls(config)
```

3. Add template retrieval (knows how to get it)

```
def _get_template(self) -> str:

    if self.config.template == TemplateType.NEW_TEMPLATE:

        return NEW_TEMPLATE_STRING

    # ...
```

Result: The class now "knows" about the new template type.

THE "SELF-AWARENESS" METAPHOR

Think of `DocumentBuilder` like a skilled craftsman:

- **Knows Their Tools:** The class knows what templates it has (like a craftsman knows their tools)
 - **Knows Their Process:** The class knows the steps to generate a document (like a craftsman knows their workflow)
 - **Knows Their Capabilities:** The class knows what it can and can't do (like a craftsman knows their limits)
 - **Self-Documenting:** The API shows what it can do (like a craftsman's workspace shows their tools)
-

SUMMARY

How DocumentBuilder "Knows" It Can Generate Documents:

- **Explicit Declaration:** `TemplateType` enum declares capabilities
- **Encapsulated Logic:** All generation logic is in the class
- **Fluent API:** Methods guide you through the process
- **Self-Documenting:** The API shows what it can do
- **Composable:** Knows how to work with collections

The "Self-Awareness" is:

- Not magic - it's good design
 - Not AI - it's encapsulation
 - Not sentient - it's explicit knowledge
 - **It's a class that knows its own capabilities and how to use them**
-

CODE EXAMPLE: THE FULL FLOW

```
from waft import DocumentBuilder
```

Step 1: Class "knows" it can create field guides

```
doc = DocumentBuilder.field_guide(
```

```
    title="My Guide",
```

```
    content="
```

INTRODUCTION

Content here.

",

```
printer_friendly=True # Class "knows" to convert
```

```
)
```

Step 2: Instance "knows" it can generate itself

```
path = doc.save("output.pdf")
```

What the class "knew" during this process:

✅ I'm a field guide (from field_guide() method)

✅ I need printer-friendly conversion (from flag)

✅ I use FIELD_GUIDE template (from config)



I render with Jinja2 (from generate() method)



I create PDF with WeasyPrint (from generate())



I save to "output.pdf" (from save() parameter)

```
print(f"Generated: {path}")
```

The class doesn't "think" - it "knows" through explicit design. Every capability is declared, every process is defined, every step is clear. That's what makes it self-aware in the programming sense.

DOCUMENT INFORMATION

Created by: Christopher Tavalazzi, Claude Opus 4.5 (Cloud), and Cursor system using multiple AI LLMs available at the time of release of Claude Opus 4.5, Gemini 3, GPT 5.2, and others

Date: 2026-01-11

Purpose: Explain how DocumentBuilder class works and how it "knows" it can generate documents